Test Summary Report: Stripe API Integration

Project: Stripe API Manual Testing (Test Mode)

QA Analyst: Paula Granado

Status: 🟢 **100% Pass Rate > Date:** March 2026

Tooling: Postman & Stripe Dashboard

Testing Types: Functional, Negative, Security, Error Handling.

---

1. Executive Summary

This report documents the functional and integration testing of the Stripe API (v1). The primary goal was to validate the stability of the **Customer Management** and **Payment Intent** lifecycles. All 12 test cases were executed successfully in Test Mode, ensuring that the API adheres to documented business rules and security constraints.

2. Project Analysis & QA Insights

This project focused on validating the core payment flow of the Stripe API using Postman. Beyond "happy path" scenarios, the testing scope included a series of critical negative tests to ensure system reliability.

The most challenging and valuable part of this project was the **negative testing**. These scenarios required careful debugging and a deep understanding of the API's error-handling mechanisms. For example, validating a declined card required troubleshooting several layers—from authentication to request body formatting—before successfully confirming the API's enforcement of the expected `402 Payment Required` status and the `card_declined` error code.

Through this process, I identified the critical importance of clear API error messages. Specific details in JSON responses, such as `resource_missing` and `payment_intent_unexpected_state`, served as a roadmap for the debugging process. This project demonstrates my ability to analyze complex API interactions and ensure that a system handles unexpected situations gracefully.

3. Testing Scope & Methodology

- **Resources Tested:** `/v1/customers` and `/v1/payment_intents`.
- **Testing Type:** API Manual Testing (Black-box).

- **Tools:** Postman (Collections & Environment Variables).
- **Authentication:** Bearer Token via Secret API Key.
- **Data Format:** `x-www-form-urlencoded`.

4. Key Findings & Validations

🤳 Customer Management (TC01 - TC06)

- **CRUD Capabilities:** The API demonstrated robust Create, Read, Update, and Delete operations.
- **Data Integrity:** Metadata and attributes (name, email) were correctly mapped and persisted.
- **Error Handling:** The API effectively blocked invalid email formats (TC02) and correctly returned `404 Not Found` for non-existent IDs (TC06).

💳 Payment Flow & Intents (TC07 - TC12)

- **State Management:** Payment Intents transitioned correctly from `requires_payment_method` to `succeeded` or `requires_confirmation`.
- **Business Rules:** The API strictly enforced mandatory fields, rejecting missing amounts (TC10) or negative values (TC11).
- **Security:** Declined cards (TC09) were accurately captured with specific error codes, providing clear feedback for the integration.

5. Detailed Test Case Log

5.1 Execution Metrics Summary

Category	Total Cases	Passed	Failed	Blocked	Success Rate
Customer Management	6	6	0	0	100%
Payment Flow (Intents)	6	6	0	0	100%
TOTAL	12	12	0	0	100%

5.2 Individual Test Execution Details

ID	Test Name	Result	Key Observation
TC01	Create Customer (Valid)	**PASS**	Status 200 OK. Unique ID generated.

TC02	Create Customer (Invalid Email)	**PASS**	Status 400. Correctly identified invalid format.
TC03	Retrieve Customer	**PASS**	Data matches the creation payload.
TC04	Delete Customer	**PASS**	Status 200. deleted: true returned.
TC05	Update Customer	**PASS**	Email successfully updated and verified via GET.
TC06	Retrieve (Non-Existent)	**PASS**	Status 404. Error message as expected.
TC07	Create Payment Intent	**PASS**	Status 200. Status: requires_payment_method.
TC08	Confirm Payment Intent	**PASS**	Status 200. Transaction processed successfully.
TC09	Confirm (Declined Card)	**PASS**	Status 402. Error code: card_declined.
TC10	Create Intent (Missing Amount)	**PASS**	Status 400. API identified missing required fields.
TC11	Create Intent (Invalid Amount)	**PASS**	Status 400. Rejected negative integer.
TC12	Confirm (Already Confirmed)	**PASS**	Status 400. Prevented duplicate processing.

6. Final Conclusion

The Stripe API demonstrated high stability and strict adherence to documented standards throughout this test cycle. All responses were consistent with **RESTful standards**, and no functional defects were identified. The system is **APPROVED for integration**, as all validation constraints and security protocols are functioning as expected, ensuring a secure and predictable environment for frontend applications.

7. Test Execution Evidence

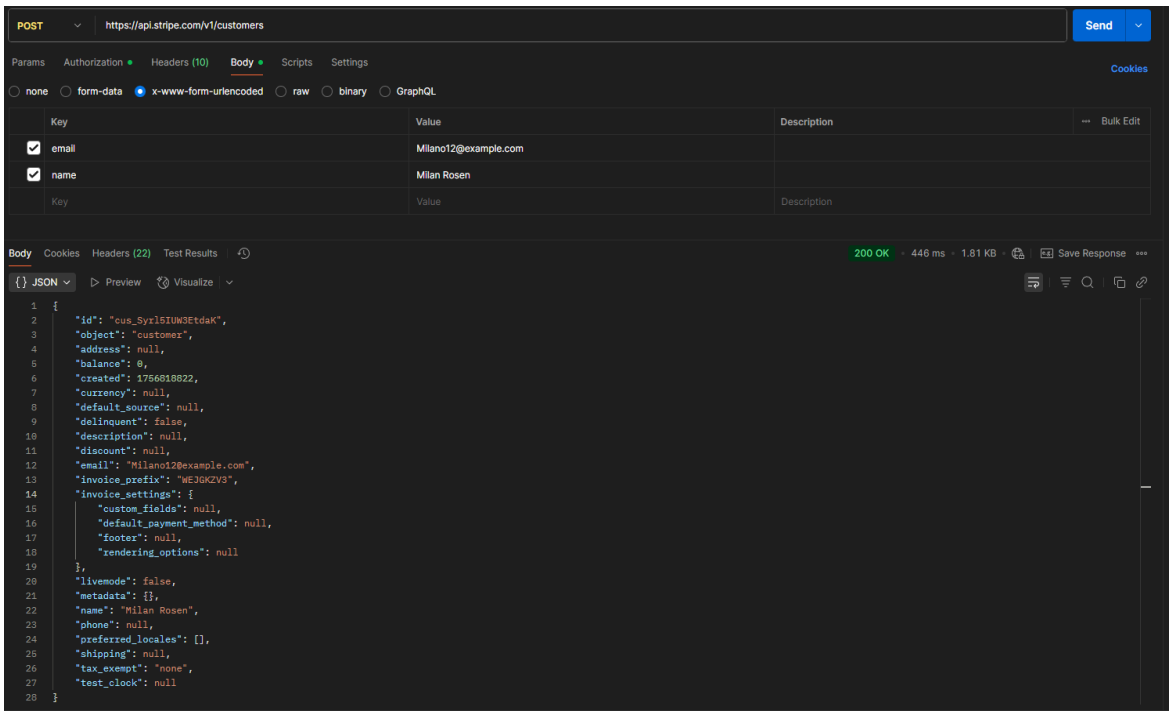
Due to the high volume of technical evidence (screenshots) and for better version control, all visual proofs of execution are hosted in the official project repository.

📂 **Full Evidence Gallery:** GitHub Repository - Stripe API Screenshots

*The repository includes organized captures of Postman Requests/Responses and Stripe Dashboard status for all **12 test scenarios**.*

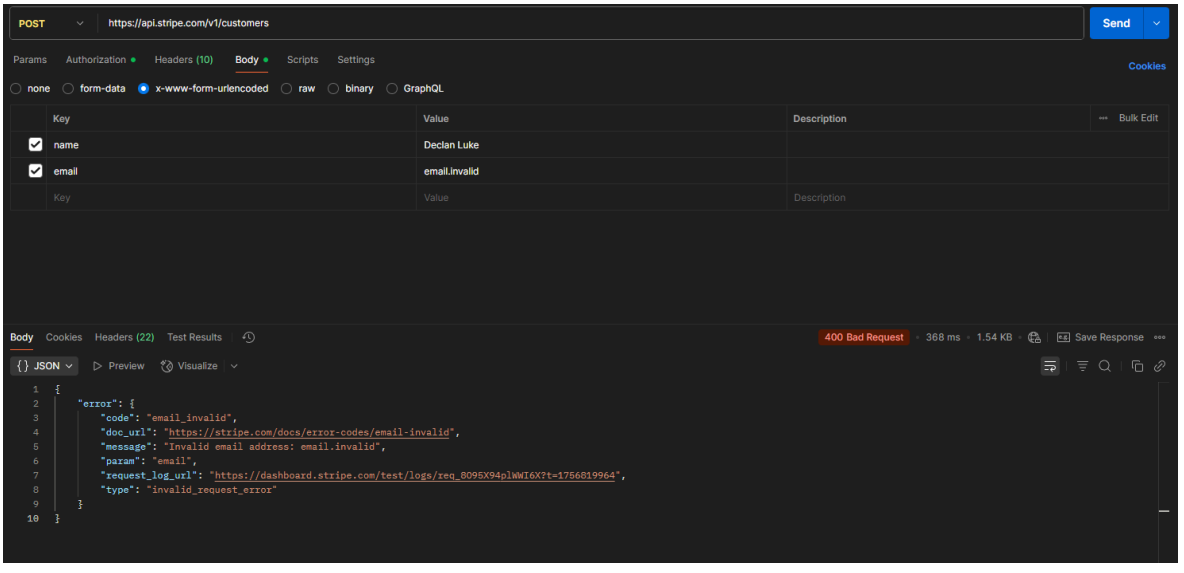
Sample Evidence (Key Scenarios):

STRP01 - Create Customer (Success)



POST request to `/v1/customers`. Evidence shows a 200 OK status and the successful generation of a unique customer_id (cus_...). Validation of name and email persistence in the JSON response.

STRP02 - Create Customer (Invalid Email)



Negative scenario validation. API correctly returned 400 Bad Request when an invalid email format was provided. Evidence confirms the error message: `invalid_email`.